

Security and the operating system

Reaching the machine from inside SD, how privileged work is done, and the audit trail.

This page continues Security.

Reaching the operating system from inside SD

There are three ways out of SD onto the machine, and all three are on one list.

	WHAT IT IS	GOVERNED BY
<code>sh</code> and <code>!</code>	a shell at the <code>:</code> prompt	<code>os.users</code> field 1
<code>OS.EXECUTE</code>	the operating system from inside a BASIC program	<code>os.users</code> field 2
<code>edit</code> and <code>micro</code>	a text editor, running outside SD	<code>os.users</code> field 2

`os.users` is in SDSYS and is read-only to everybody else on disk. That ACL is the whole of the protection: without it a user grants themselves any of the three in one line, and every check above it is decoration.

Four rules hold for all three:

- 1. A missing record, or a missing file, means no.** An installation that has never set `os.users` up denies all three to ordinary accounts. That is the opposite of the VOC tier lists, where a missing record means the *full* VOC — do not carry one convention across to the other.
- 2. An elevated session passes on its own,** whatever the list says, so an empty list cannot lock the machine's own administrator out.

An ssh session *can* be elevated. Windows' OpenSSH runs as a system service and builds the logon token itself, so a member of `Administrators` arriving over ssh is handed a **full** token — not the filtered one a local sign-in would produce, and with nobody asked to consent.

It is why SD now refuses an administrator any session that did not come from this computer. Over ssh **on this machine** an administrator is still elevated, and still passes this rule without needing a record.

- 3. An API session is never treated as an administrator — which is not the same as having no way out to the machine.** An API session never holds administrator rights and cannot reach SDSYS. But this list is consulted **by user name**, and it is consulted whether or not a session is elevated: an account whose `os.users` record says `yes` gets `sh`

and `OS.EXECUTE` over the API exactly as it would anywhere else.

Every administrator has such a record, written when the account is created and refused to `modify.account os-off`. So an administrator's API session can genuinely run operating-system commands on the server — which is why an administrator is refused an API connection from any other computer. From this machine it works, and that is deliberate.

4. Nothing in SD limits what you then do. Once field 1 or field 2 says `yes`, SD is not standing between that person and the machine — **the boundary from that point on is their own Windows account's permissions**, and nothing else. A listed person gets a real shell, with pipes and redirection; an editor can open any file they are allowed to open, in the data tree or outside it.

So field 1 and field 2 are statements of trust in a person. They are not a convenience to be handed out because somebody asked for a full-screen editor. Grant them on the same basis you would grant a shell account on the machine, because that is close to what you are granting.

The record format, and how to grant and remove it, are on Administrator commands.

Privileged work is done through a script, not a command line

When SD creates an account, sets a password or edits a group, it writes a short script to a file and runs it, rather than putting a password on a command line where any local user could read it through Task Manager or WMI.

Those scripts go in `PSTMP`, a directory of its own inside the database directory, where each file belongs to the session that wrote it. Other SD users cannot read or change them.

Before that they went where every SD user could write, and the second consequence was the serious one: **another SD user could replace the script between SD writing it and SD running it, and their version would run with full administrator privilege.**

If `PSTMP` is missing, SD refuses the privileged work rather than falling back to the old location. You see the command fail rather than quietly running unprotected.

The audit trail

`C:\ProgramData\SD\sdsys\audit` records **every login, every refused login, every `logto`, every refused `logto`, and every `grant` and `revoke`**, with date, time and the Windows user it belonged to.

```
2026-08-16 11:42:07 user=don uid=1 pid=8624 LOGTO account=SDSYS
```

The refusals are the interesting half. An entry saying somebody who is not an administrator asked for SDSYS by name, or asked for an account they have not been granted, is the thing worth seeing. **Failed API logins are recorded too, with the reason.**

This is not the error log and does not behave like it. The error log throws away its oldest half when it fills; the audit file is **renamed with the date and time and a new one started, so nothing is ever discarded.** Removing the old ones is your decision — SD will not do it for you, and they will accumulate.

SD USERS CAN	SD USERS CANNOT
add to it	read it, change a record in it, empty it, rename or delete it

Windows itself refuses, not SD, so a user cannot quietly remove the line that records what they did. Administrators can read it and do anything else to it as well: this raises the floor against ordinary users rather than trying to constrain somebody who owns the machine.

Windows keeps its own separate record of account and group changes in the Security event log. **Read the two together.**

What is still not true

SD users are not isolated from each other's data at the file level, beyond the Per-account directory lock. Everyone who uses SD needs file access to the tree, because their own process does the I/O. Anyone deploying SD for ten people over ssh should be told that plainly.

SD has no file-level access control of its own on the console and ssh paths. The one place a path gate does exist is the API, where a session is confined to the account it stands in — see API access.